
BHASHALENS ARCHITECTURE BLUEPRINT

A Privacy-First, Offline-Capable, Bidirectional Translation & Assisted Language Learning Platform for Indian Vernaculars

Version: 1.0.0 (Production Blueprint)

Date: May 2026

Author: BhashaLens Technical Development Team

Target Event: HackIndia AI Agents Hackathon 2026

1. Project Vision

BhashaLens is designed to break language barriers across the Indian subcontinent by providing a **100% offline, privacy-first, and highly-optimized machine translation and language assistance suite**. Unlike typical translation apps that rely on persistent cloud servers and constant network handshakes, BhashaLens operates strictly on-device using quantized Neural Machine Translation (NMT) and assisted language technologies. This architecture ensures complete user privacy, guarantees zero data leakage of personal text history, and enables seamless functionality in regions with poor, intermittent, or completely absent internet connectivity.

The platform revolves around four critical pillars:

- **Absolute Offline Privacy:** All database logs, caching systems, and translation weights reside strictly inside the sandboxed device memory. Plaintext PII is never transmitted.
- **Direct Bidirectional Vernacular Translation:** Implements optimized direct paths between Hindi, Marathi, and English, avoiding intermediate English pivoting. This cuts inference latency in half and preserves localized semantics.
- **Dynamic Online-Offline Hybrid Routing:** Automatically shifts from low-latency online models (e.g., Google Gemini AI API) to local optimized engines (TFLite INT8) during network disruption or latency spikes (>1500ms).
- **Universal Digital Accessibility:** Custom screen reader integration, tactile haptic cues, and speech-controlled Voice Navigation Controllers tailored for users with visual and cognitive impairments.

2. System Architecture

BhashaLens uses a decoupled monorepo architecture divided into four functional component layers. This strategic separation ensures massive scalability, ease of local testing, and robust failover paths.

Component / Layer	Focus	Primary Technologies
bhashalens_app (Mobile & Desktop)	UI render loop, offline isolate translation interpreter, local encrypted storage, dynamic S3 package loader.	Flutter SDK v3.19+, Dart, tflite_flutter, SQLite, Hive (AES-256)
functions (Cloud Engine)	Serverless python backend triggers, user synchronization gateways, analytics aggregation, cloud metrics logs.	Python 3.10+, Cloud Functions for Firebase, Firestore
ml_pipeline (ML Compilation)	Vocabulary pruning, knowledge distillation from large teachers, float32 to INT8 quantization, and format compilation.	PyTorch, HF Transformers, CTranslate2, SentencePiece, TFLite
infrastructure (Provisioning)	Infrastructure-as-code scripts, cloud API gateways, low-latency REST route Lambda nodes, unified deployment.	Terraform v1.6+, AWS Lambda, AWS S3, Firebase CLI

The Smart Hybrid Translation Flow:

1. User initiates translation → App triggers the connection-aware smart_hybrid_router.dart.
2. *Online Path:* If network is high-speed, requests route to the Cloud API (Google Gemini AI), executing context-aware translation, returning the response, and caching it in SQLite.

3. *Offline Path*: If offline or network latency exceeds 1500ms, the system intercepts the request → queries the AES-256 local database cache → returns instantly on hit (<30ms).
4. *Local Inference*: On cache miss, requests trigger the TFLite background Isolate engine to perform neural network forward pass, bypassing the main thread to ensure a solid 60 FPS UI rendering.
5. *On-Demand Lifecycle*: If the required localized language pack is absent, the user is prompted to dynamically fetch the zip model pack (~78MB) from AWS S3, verify the SHA-256 hash, and unpack.

3. Technology Stack

BhashaLens leverages premium, modern frameworks across all operational layers to deliver enterprise-grade performance.

Operational Layer	Core Frameworks & Tools	Key Purpose in BhashaLens
Mobile Frontend	- Flutter SDK (v3.19+) - Dart Language	Cross-platform layout rendering, unified codebase, high-performance isolates.
On-Device Inference	- tflite_flutter (v0.10.4) - tflite_flutter_helper (v0.3.1)	Low-level tensor execution of INT8 models directly on mobile CPU/NPU.
Local Secure Storage	- SQLite & Hive DB - Encrypted Box (AES-256)	Secure localized translation caching, historic telemetry logs, user preference files.
Serverless Backend	- Python 3.10+ & Firebase - AWS Lambda & API Gateway	High-speed, scalable REST endpoints, fallback transcription, profile synchronization.
ML Training & Quant	- PyTorch & HF Transformers - CTranslate2 & SentencePiece	Knowledge distillation, model vocabulary pruning, sub-15MB INT8 quantization.
Infra as Code (IaC)	- Terraform (v1.6+) - AWS S3 & CloudFront	Automated provisioning of storage buckets, global model distribution, edge servers.

4. Model Distillation & Quantization Pipeline

Deploying massive Transformers directly to mobile devices is prohibited due to strict RAM constraints and computational limits. BhashaLens solves this through a systematic model shrinking pipeline:

The Four-Step Optimization Workflow:

- Knowledge Distillation:** Transfers language comprehension from a massive Teacher network (e.g., NLLB-200 distilled 600M parameters, 300MB FP32) into a highly compressed Student Marian architecture (25.4M parameters).
- Vocabulary Pruning:** Strips non-Indic tokens out of the model's massive SentencePiece vocabulary (reducing vocabulary from 256,000 down to 32,000). Slicing the embedding matrices reduces parameters and shrinks the file size by ~70%.
- Weight Slicing:** Reduces hidden dimensions (d_model set to 256) and layer counts (6 encoder, 6 decoder layers), creating a lightweight student model (~101.6MB in original FP32).
- Post-Training Quantization (INT8):** Maps weight matrices from 32-bit floating point (FP32) to 8-bit integer (INT8) representation. Shrinks final weight footprint down to a tiny **14.8MB**, matching our sub-15MB deployment goal while preserving translation quality (BLEU score remains > 25).

The Dual Distillation Loss Formula:

The distilled student is trained utilizing a combined loss function combining standard cross-entropy and soft-target KL-Divergence. This enables the student model to mimic the soft probabilities of the teacher, retaining vocabulary nuances:

```

# Combined Distillation Loss Definition in PyTorch
class DistillationLoss(nn.Module):
    def __init__(self, alpha=0.3, temperature=4.0):
        super().__init__()
        self.alpha = alpha
        self.temperature = temperature
        self.ce_loss = nn.CrossEntropyLoss(ignore_index=-100)
        self.kl_loss = nn.KLDivLoss(reduction="batchmean")

    def forward(self, student_logits, teacher_logits, labels):
        # 1. Hard Target Cross-Entropy Loss
        loss_ce = self.ce_loss(student_logits.view(-1, student_logits.size(-1)), labels.view(-1))

        # 2. Soft Target KL-Divergence Loss
        p_teacher = F.softmax(teacher_logits / self.temperature, dim=-1)
        log_p_student = F.log_softmax(student_logits / self.temperature, dim=-1)

        loss_kl = self.kl_loss(
            log_p_student.view(-1, log_p_student.size(-1)),
            p_teacher.view(-1, p_teacher.size(-1))
        ) * (self.temperature ** 2)

        # Total Weighted Loss Combination
        total_loss = (self.alpha * loss_ce) + ((1.0 - self.alpha) * loss_kl)
        return total_loss

```

5. IndicTrans2 Setup

BhashaLens integrates AI4Bharat's state-of-the-art **IndicTrans2** framework to achieve superior translation accuracy. Native scripts and quantization mappings are structured to harness the IndicTrans2 vocabulary mapping, guaranteeing proper handling of regional scripts (Devanagari, Tamil, Telugu, etc.).

- **Indic Script Tokenizer:** Uses a customized SentencePiece processor trained specifically on massive Indic corpora to handle complex script graphemes.
- **Vocabulary Extraction:** We run a vocabulary pruner script to isolate specific language pairs (e.g., Hindi and Marathi). Unneeded tokens from the original 256,000 vocabulary size are completely pruned out, leaving a clean 32,000 token mapping table.
- **Script Normalization:** Includes a pre-processing pipeline that normalizes Unicode script variations in Devanagari (e.g., Marathi-specific nukta and vowel marks) to avoid Out-Of-Vocabulary (OOV) errors.

6. ONNX Model Workflow

In addition to TFLite, BhashaLens supports a highly-efficient cross-platform **ONNX Runtime (ORT)** inference pipeline for desktop targets (Windows, macOS) and high-end mobile processors:

ONNX Pipeline Step	Actions Performed	Tools Used
1. PyTorch Export	Distilled PyTorch student model exported to static computation graph (.onnx).	torch.onnx.export
2. Graph Fusion	Optimizes computation nodes, performs constant folding, fuses adjacent LayerNorm and Attention operations.	ORT Graph Optimizer
3. INT8 Quantization	Quantizes weights dynamically to INT8, dramatically reducing model size.	onnxruntime.quantization
4. Native Execution	Executes via ONNX Runtime mobile package with hardware acceleration (NNAPI/CoreML).	ONNX Runtime Mobile SDK

7. Parameter-Efficient Adapter Strategy (LoRA)

To support multiple language pairs without multiplying model sizes, BhashaLens introduces a **Low-Rank Adaptation (LoRA) strategy**. Instead of compiling distinct 15MB models for each pair (e.g., hi-en, mr-hi, en-mr), we use a unified base model and swap language-specific adapters:

- **Shared Base weights:** A single distilled, pruned student model acts as the foundational translation base, residing permanently in memory (~15MB).
- **Lightweight Adapters:** LoRA adapters are injected into the key attention weight projection matrices (W_q and W_v). These adapters contain low-rank weights (A and B) specifically trained for target language direction sets.
- **Dynamic Hot-Swapping:** When the user switches translation direction from Hindi → English to Marathi → Hindi, the base model remains loaded in memory. The system simply unbinds the old adapter and swaps in the new adapter in real-time. Since each adapter file size is < **1MB**, RAM utilization is kept ultra-lean, fully

avoiding out-of-memory crashes.

8. Flutter Clean Architecture Structure

The Flutter mobile codebase (bhashalens_app) complies strictly with Clean Architecture and Domain-Driven Design (DDD) patterns. It is split into logical modules keeping rendering loops decoupled from heavy model execution:

```
bhashalens_app/
■■■■ lib/
■   ■■■■ core/                # Global Shared Resources
■   ■   ■■■■ theme/           # App palettes, modern typography system
■   ■   ■■■■ database/        # AES-256 Encrypted SQLite & Hive setup
■   ■   ■■■■ accessibility/   # Screen reader hook controls & tactile haptics
■   ■
■   ■■■■ models/              # Pure Domain Data Structures
■   ■   ■■■■ language_pair.dart # Language enum and source/target configurations
■   ■   ■■■■ translation_result.dart # Return capsule: text, confidence, latency, source
■   ■   ■■■■ translation_history_entry.dart # Storage capsule structure for histories
■   ■
■   ■■■■ services/            # Business Logic Layer
■   ■   ■■■■ translation_engine.dart # Abstract interface for NMT back-ends
■   ■   ■■■■ tflite_translation_engine.dart # Low-level TFLite INT8 compiler interpreter
■   ■   ■■■■ offline_translation_service.dart # Cache-first controller with encryption
■   ■   ■■■■ smart_hybrid_router.dart # Network latency-aware online-offline switcher
■   ■   ■■■■ encrypted_local_storage.dart # AES-256 secure cache manager
■   ■
■   ■■■■ features/            # Cohesive App UI Features
■   ■   ■■■■ splash/           # Bootstrap and background initialization
■   ■   ■■■■ auth/             # Secure local auth profiles
■   ■   ■■■■ translation/      # Main interactive translation UI
■   ■   ■■■■ explain/          # Context explainers for grammar correction
■■■■ assets/models/           # Default compiled local INT8 models
```

9. Prompt Engineering & LLM Integration

For hybrid online operations, BhashaLens leverages Google Gemini AI models. Prompts are strictly engineered to enforce output formats, returning deterministic structures that can be parsed reliably by the client application. Our prompts are optimized for translation, grammar correction, and dialogue practice:

Translation API Prompt:

Translate from [source_language] to [target_language]. Output only the translation, no extra text.\n\n[Text]

Grammar Correction Prompt (Structured JSON Enforced):

You are a [language] expert. Check for grammar errors. Format as JSON with \"corrected_text\" and \"corrections\" list of dicts (original, corrected, explanation):\n\n[Text]

10. Dataset Sources

To ensure premium translation quality, BhashaLens models are trained and distilled using high-fidelity, publicly-recognized datasets across multiple domains:

Dataset Name	Primary Focus & Language Pairs	Size / Sample Scale Used
Samanantar (AI4Bharat)	Largest parallel Indic corpus. Mainstay for Hindi-English, Marathi-English and direct Hindi-Marathi pairs.	~5M parallel lines per pair selected.
IIT Bombay Corpus	High-quality parallel corpus optimized for English-Hindi translation. Covers general domain texts.	~1.6M clean sentence pairs.
PMIndia Parallel	Parallel sentences extracted from official government portals. Highly formal vocabulary and grammar.	~50k sentence pairs per language.
FLORES-200	Multi-way benchmark parallel evaluation set. Used strictly for BLEU and model validation checks.	100% of standard evaluation dev/test sets.
Mixed Synthetic Data (hi_mr_synthetic.jsonl)	High-fidelity, LLM-generated dialogue items in Hindi and Marathi representing local conversational context.	~1,000 highly targeted conversational dialogs.
Mono Hindi / Marathi	Monolingual clean corpora (e.g., mono_hi.txt) utilized for training SentencePiece tokenizers and vocabulary pruning.	~10MB raw text corpora files.

11. Core System Performance Goals

BhashaLens sets a high standard for performance, stability, and efficiency. Every subsystem is mapped to clear, measurable engineering targets:

Operational Metric	Target Goal Limit	Architectural Enforcement Mechanism
Model Size	Sub-15 MB per language pair pack	Vocabulary pruning (32k) + embedding table slicing + dynamic INT8 post-training quantization.
Translation Latency	Sub-1 second text processing	Quantized matrix multiplication, background execution isolates, cache-first SQLite database hit bypass.
Model Loading Time	Sub-2 seconds initialization	Memory mapping TFLite files dynamically, avoiding ahead-of-time bulk array loading.
UI Smoothness	Consistent 60 FPS frame rate	Isolate-based background thread execution, protecting the main Flutter UI thread from heavy calculations.
Data Security	Zero plaintext leak of PII history	On-device sandboxed storage protected under industry-standard AES-256 database encryption.
Model Integrity	100% corruption recovery	Automated SHA-256 checksum validation upon streaming download packages, with auto-retry recovery.

12. Future Roadmap

BhashaLens is structured to expand seamlessly. The future roadmap details key milestones designed to increase functional coverage while maintaining performance boundaries:

Phase 1: Foundations (Current)

- Complete direct, bidirectional translation for English ↔ Hindi ↔ Marathi.
- Validate background Isolate execution loops and verify the secure AES-256 SQLite caching system.
- Deploy Firebase Cloud Functions backend and Terraform AWS S3 distribution buckets.

Phase 2: Regional Expansion (Q3 2026)

- Add model packs for Tamil, Telugu, and Bengali following identical distillation pipelines.
- Implement offline, lightweight script-based language identification (sub-10ms classification).
- Introduce streaming speech-to-speech translation by chaining mobile-optimized Whispers with NMT.

Phase 3: Deep Optimization (Q1 2027)

- Transition from 8-bit to 4-bit integer quantization, dropping model size to sub-8MB while keeping BLEU > 24.
- Implement dynamic GPU hardware acceleration hooks via Vulkan and Metal in tflite_flutter.
- Integrate on-device federated learning and reinforcement learning from user feedback (RLHF) loops, updating weights privately.